

COMS W4156 Advanced Software Engineering (ASE)

November 21, 2023

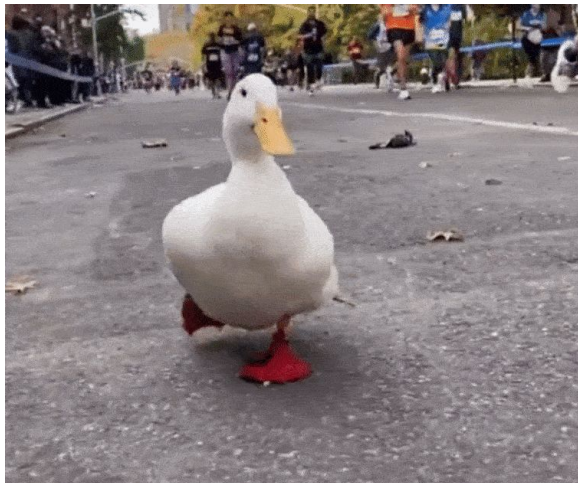


Agenda

1. Test Order Overview
2. Dependencies and Flaky Tests
3. State Transition Testing



Test Runners Run Tests



A test runner executes a test suite (a set of tests) and reports results - which tests passed and which tests failed

Originated with unit testing but not restricted to unit testing, test runners can run any kinds of tests where code calls other code

Usually does not stop when a test fails - instead runs all the tests and reports all the results

? When might you want a test runner to stop after the first failure ?

Test Runners Run Tests In Some Order

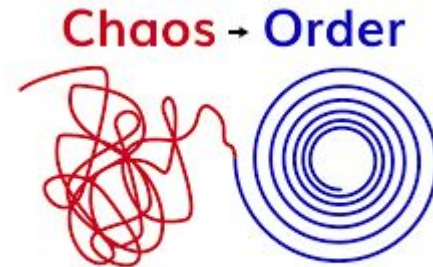
Test runners typically have a built-in default [test order](#), e.g., alphanumeric, most recently created, most recently modified, random, etc.

Note alphanumeric order means test14 runs before test2

Different tools, [even different versions of same tool](#), may run tests in different default orders

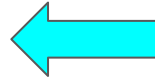
Most test runners provides annotations (e.g., [@TestMethodOrder](#)) to optionally specify order

Some tools run multiple tests in parallel (e.g., [CloudBees Build Acceleration](#)), which could break any relied-upon ordering scheme

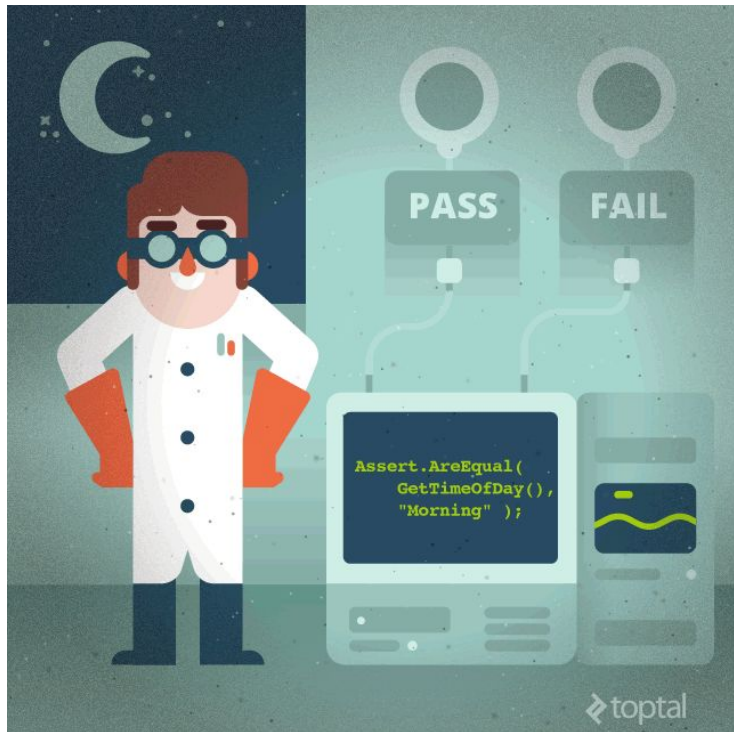


Why Does Test Order Matter?

- Dependencies and Flaky Tests
- State Transition Testing



Some Tests Are “Flaky”



A [flaky test](#) is a test that produces different results (pass or fail) on different test suite executions *even when no code has changed*

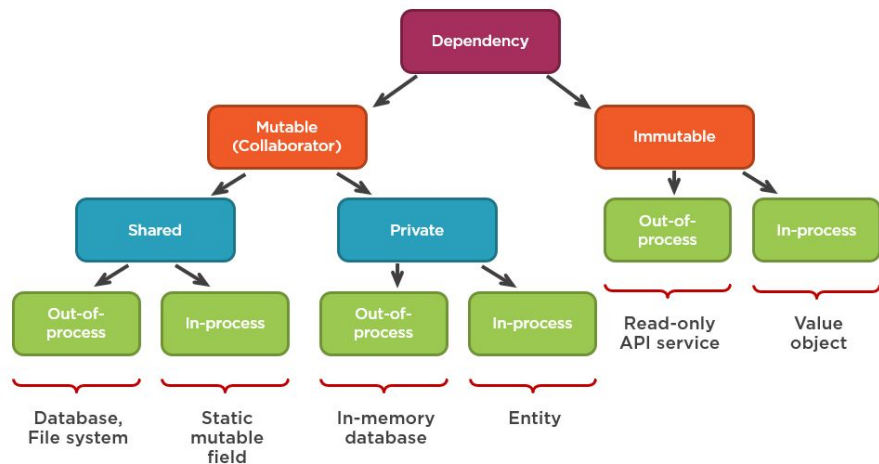
Causes include:

- [hidden dependencies](#) among tests that cause different results for different test orders,
- reliance on external resources with non-deterministic latency and availability (mitigated by [mocking](#)), or
- [non-determinism](#) in the test's own code or in the unit under test

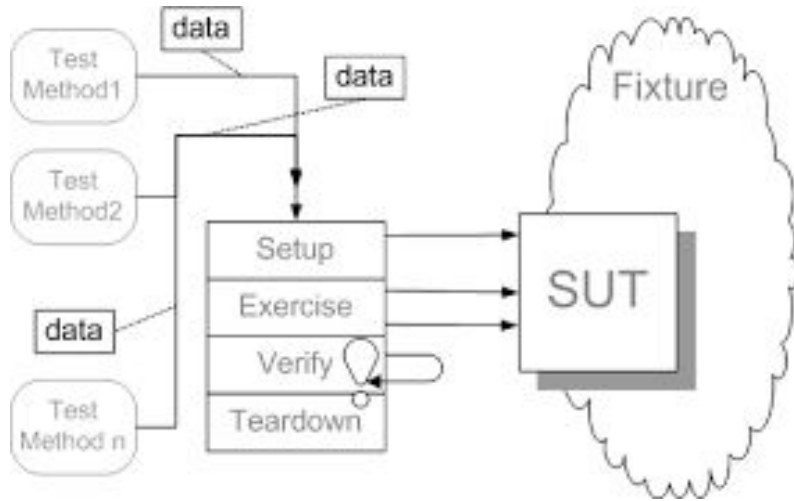
Dependencies Among Tests

Tests *intended* to be dependent on each other (e.g., [shared state](#) and/or one test acts as setup or teardown for another), and executed in a specified order, should be placed in the same “[test class](#)”

Application code [necessarily has dependencies](#) on other application code and often on third-party code, and tests are necessarily dependent on the application code they test, but test classes are supposed to be *independent* from each other - but often aren't



Test Classes



Testing frameworks treat [test classes](#) as an atomic unit, all or nothing

The state shared within a test class might include class or global variables, test doubles and/or real resources

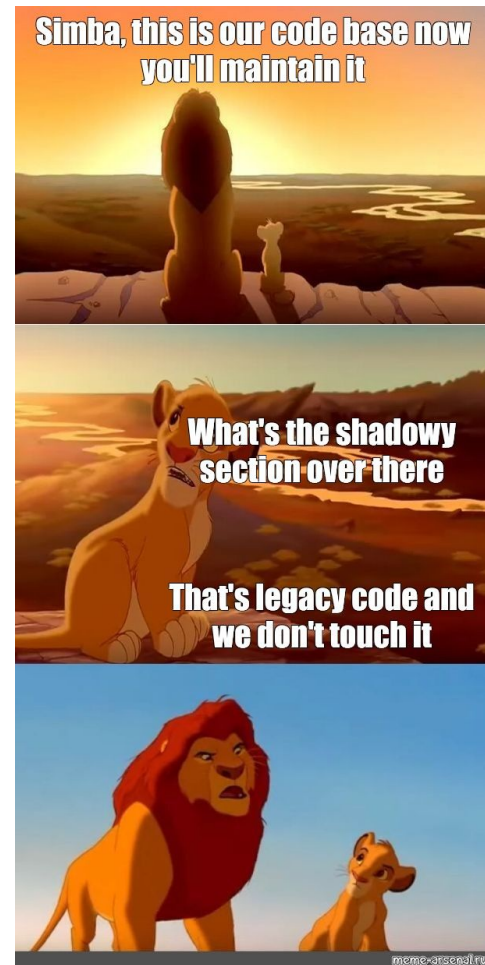
Setup/teardown [fixtures](#) associated with test classes construct and clean up the shared state before/after entire test class and each test in the class

Hidden Dependencies Among Test Classes

Finding all dependencies among test classes is expensive: Run all subsets of the test suite in all possible orders to detect discrepancies. For n test classes, in worst case costs $O(2^n)$, but only has to be done once - until some test class changes. Note this assumes no tests are flaky for other reasons and does not address [multi-threading race conditions](#)

Rather than try to find all dependencies, some testing tools offer an option to *prevent* dependencies from affecting test results by restarting/rebooting the runtime environment in between test classes - just in case

Extremely expensive to reinitialize before each test class: It might take 3-5ms to run a JUnit test class vs. 1.4s to restart JVM between each pair of test classes. It takes much longer to reboot Linux. But, technically, only $O(n)$ cost for n test classes...



Why Should Test Classes Be Independent?

Some test runners intentionally run tests in a hard-to-predict order

Newer versions of test runners may run tests in a different order than older versions (upgrading the unit testing framework causes previously passing tests to fail)

Test suites finish faster when test classes run in parallel with each other

Test suites finish faster when test classes unaffected by recent changes are omitted ([impact analysis](#))



Example Re-Order



set up global state then run tests in sequence

do lots of tests;

```
test1 () { do something that changes  
global state; };
```

do lots of other tests;

```
test2 () { do something that uses  
global state; }
```

set up global state then run tests in sequence

do lots of tests;

```
test2 () { do something that uses  
global state; }
```

do lots of other tests;

```
test1 () { do something that changes  
global state; }
```

Example Parallel

set up global state then run tests in parallel;

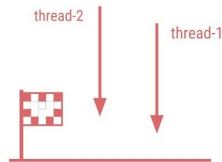
do lots of tests;

```
test1 () { do something that changes  
global state; }
```

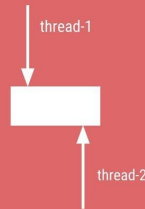
do lots of other tests;

```
test2 () { do something that uses  
global state; }
```

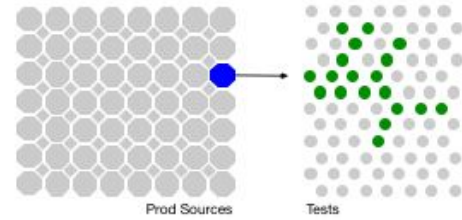
Race Conditions



Data Races



How Do We Know When a Test is Affected?



If the test itself is newly added or changed, if it tests newly added or changed code, or if it accesses state accessed by newly added or changed code, then it's affected

Otherwise, use [test impact analysis](#) to determine those tests whose results might change due to recently added or changed code - and those tests whose results cannot change

The goal is to reduce the number of tests to run by identifying those most likely to detect a newly added bug - which must be in the newly added code, in an interface between the newly added code and some existing code, or in some existing code that shares state with the newly changed code

How to Eliminate Flaky Tests

Ideally, all flaky tests should be tracked down and removed, but this is hard!



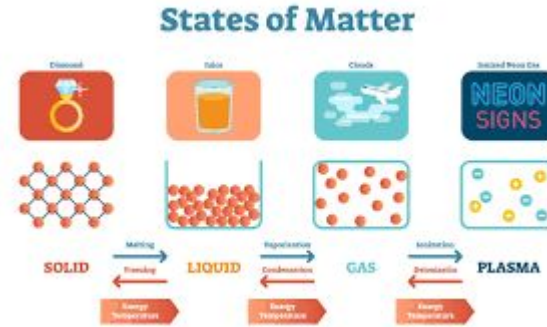
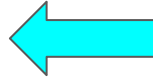
Write test assertions to accommodate application non-determinism, e.g., Kotest

Use record/replay and time-travel debugging, e.g., rr, replay, to identify root causes of non-determinism

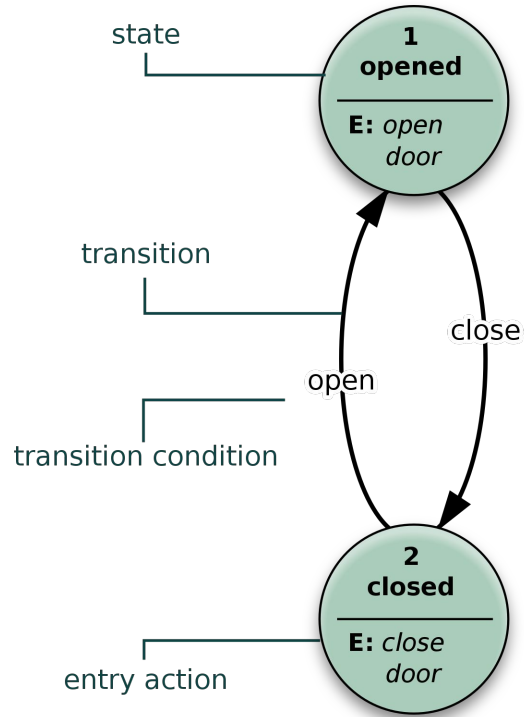
Remove all sources of non-determinism that can be removed, e.g., use re-initialized test doubles for all external resources except during explicit integration/system testing with those external resources and during end-to-end testing - when flaky tests are unavoidable

When/Why Does Test Order Matter?

- Dependencies and Flaky Tests
- State Transition Testing



State Transition Testing



Most often applied to OO classes where methods share state, typically object and class fields

Also applicable to non-OO languages when state is scoped to modular units that can be tested in isolation from rest of program

Does not scale well to large systems with many states and/or loops among states

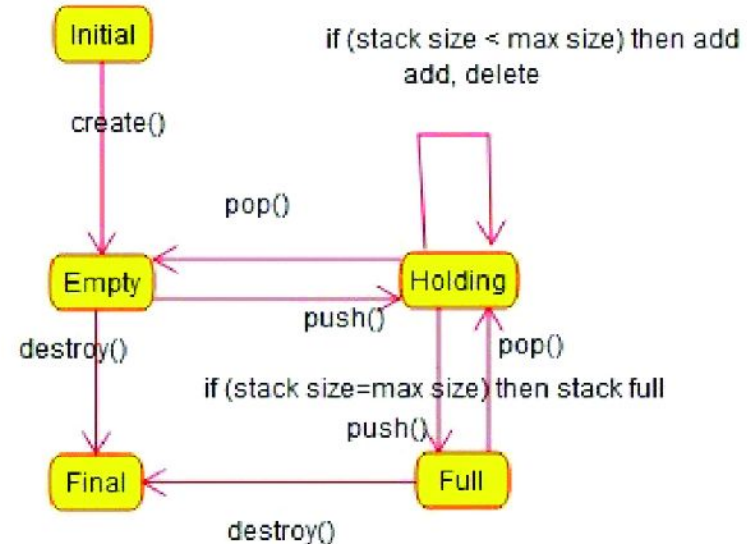
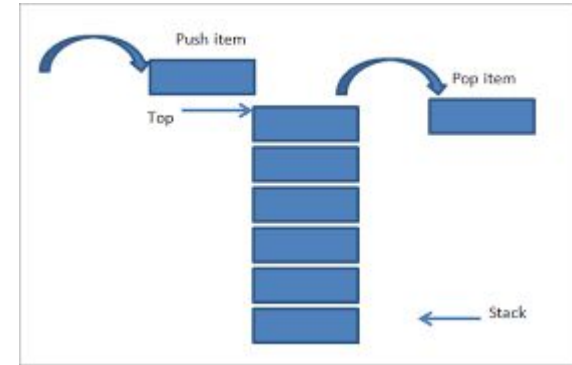
shared	mutable	state
		
this value can be used in many parts of the program	this value can be modified in place	this value is stored in one place and can be accessed

Test All Transitions in the State Diagram

Draw “[state diagram](#)” (finite state automaton) to visualize the possible states and their transitions

Choose combinations of methods and inputs to those methods that exercise all the transitions between states - which is much simpler if we can specify test order

There may be constraints on some transitions that also impact test order



Example: Stack Data Structure - Good Order

Test 1: create()

Test 2: destroy()

Test 3: create() - need new one

Test 4: push(xxx)

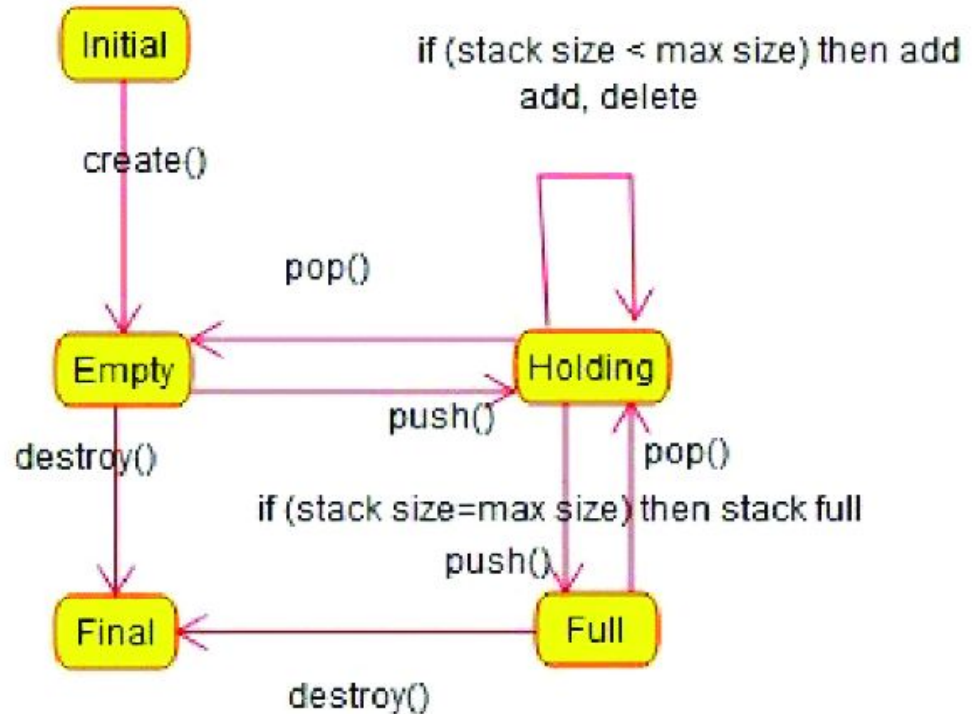
Test 5: pop() - did we get xxx?

Test 6: push(yyy)

Test 7: push(zzz)

Test 8: pop() - did we get zzz?

And so on...



Example: Stack Data Structure - Bad Order (Alphabetical)

create()

create() - need new one

destroy()

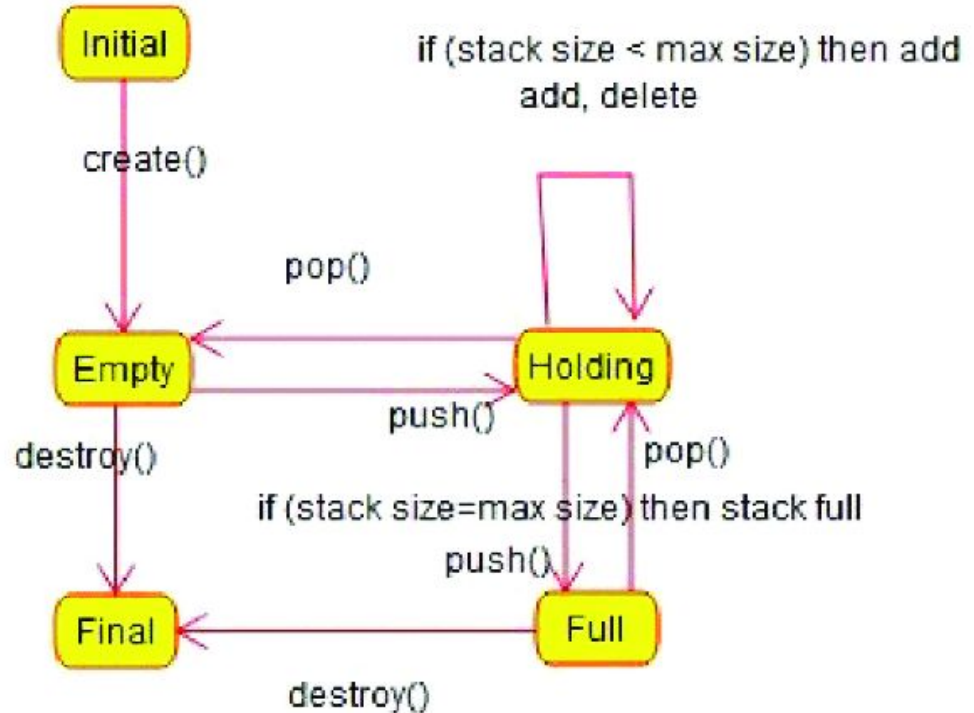
pop() - did we get xxx?

pop() - did we get zzz?

push(xxx)

push(yyy)

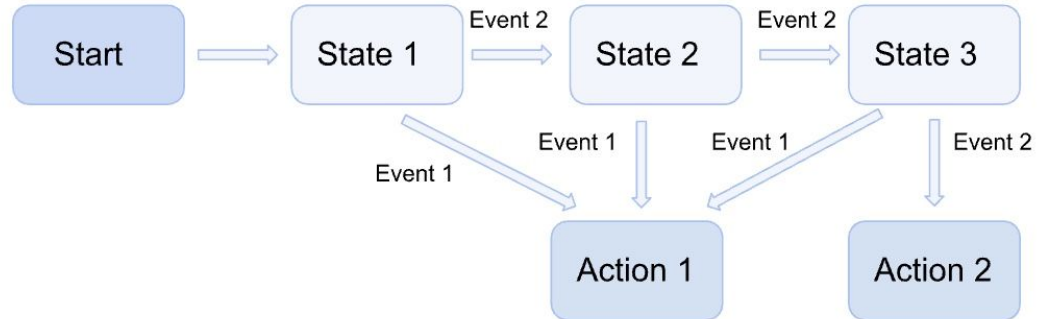
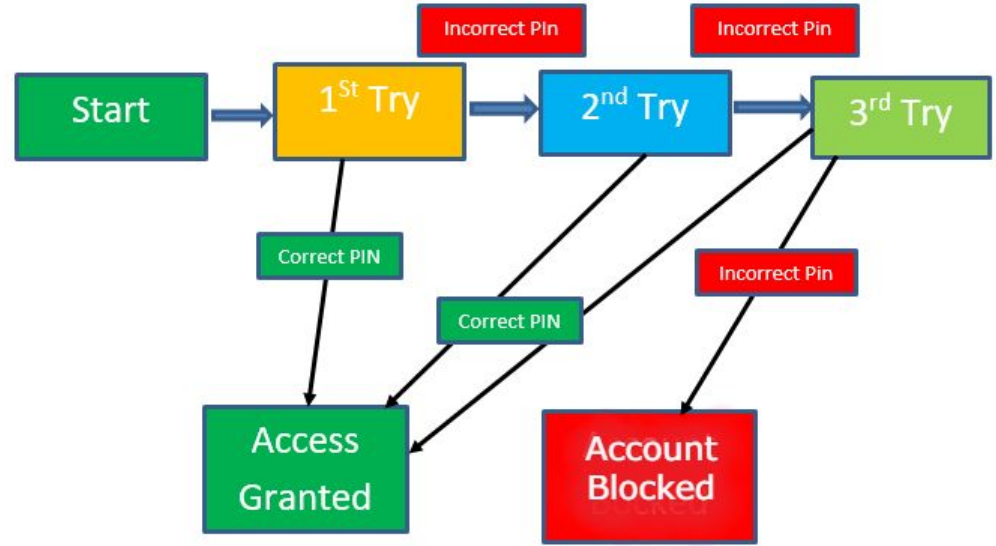
push(zzz)



Another Example: Login

? What are the appropriate tests and how should they be ordered ?

Start	Event 1	Event 2
State 1	Action 1	State 2
State 2	Action 1	State 3
State 3	Action 1	Action 2
Action 1	—	—
Action 2	—	—



Remaining Assignments

- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm

Second Assessment will be available Friday December 8 at 12:01am and due Monday December 11 at 11:59pm (4 days minus two minutes)

[Demo Day](#) is Monday, December 18, 10am-4pm, signup sheet is [here](#)



Happy Thanksgiving!

No class Thursday = Thanksgiving



Ask Me Anything

If you have questions about your team projects,
now would be a really good time to ask!